

Kora — Next Migration Plan (Phase 2 onward)

Where you actually are (verified against the code)

Phase 1 is **essentially done and it compiles clean** (`tsc --noEmit` passes). Confirmed by reading the repo:

- ✓ Electron fully stripped — no forge/main/preload anywhere.
- ✓ Plain Vite + React 19 + TypeScript, Firebase 11 SDK wired (`firebase.ts` init for auth/firestore/storage + Google provider).
- ✓ `api.service.ts` genuinely converted to Firestore (125 Firestore calls; the only "http" left is a code comment).
- ✓ **Both plan decisions landed correctly:** enums are string-based (`Approved = "approved"`) with a display-label mapping layer; IDs are strings.
- ✓ Leave approve-and-decrement uses `runTransaction` — the integrity risk was handled.
- ✓ Security rules are real and role-based (`isAdmin()`, `isSelfEmployee()`, custom claims) — not permissive stubs.
- ✓ Seed script, Firestore indexes, Storage rules, and Firebase Hosting all configured.
- ✓ Cloudinary kept, as planned.
- ✓ Git history matches the plan's step order (scaffold → types → data layer → wire+de-Electron → rules/seed).

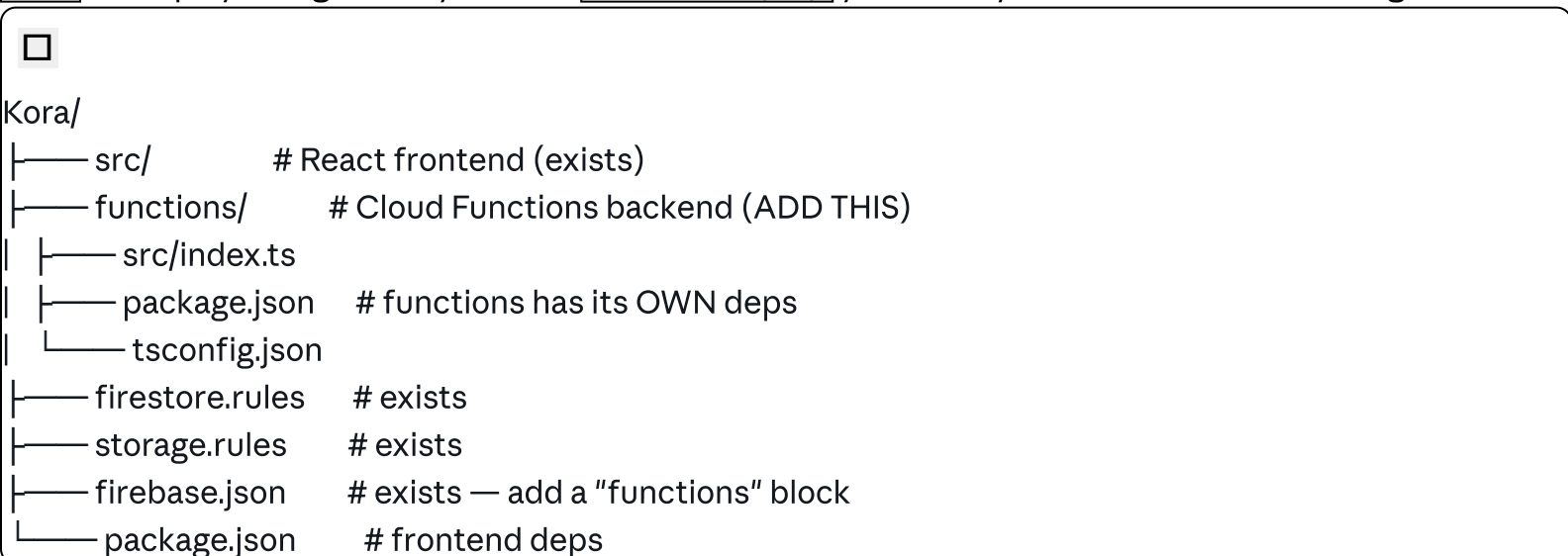
This is high-quality execution. What remains is **not** finishing phase 1 — it's verifying it against a live project, then the deferred phases.

Known gaps (all expected at this stage):

- ⚠ Zero tests (the old Jest suite was dropped, not ported).
- ⚠ Employee pages have almost no responsive work yet (1 file with breakpoints) — that's phase 4, the mobile goal.
- ⚠ 76 MB `old-coriander-code/` reference folder still committed — cleanup candidate.
- ? Not yet verified against a real Firebase project (rules/indexes/seed are written but need a live run).

Your request: backend + frontend in ONE repo

Good news — **you already have a monorepo, it's just frontend-only so far**. Firebase's model fits this naturally. "Backend in the same repo" means adding a `functions/` directory (Cloud Functions) alongside `src/`, all deployed together by the one `firebase deploy` you already have wired. Structure target:



Important nuance: `functions/` has its **own** `package.json` and `node_modules` — Cloud Functions run server-side on Node, separate from the browser bundle. This is normal and correct; don't try to share one dependency tree.

Do you even need `functions/` yet? For phase-1 parity — no, the client-side fan-out covers it. Add the backend when you hit one of these real needs (all flagged in the original plan):

Setting custom claims (role/employeeId) — this genuinely *requires* a backend; the client can't set its own auth claims securely. If admins link employees and that must set a `role` claim, you need this first. **This is the most likely reason you'll add `functions/` soon.**

Admin dashboard aggregates done server-side instead of reading many docs.

Email verification with codes (if you want the old 6-digit UX back instead of Firebase's link).

Anything that must be trusted/atomic beyond what a single client transaction gives.

Recommended phase order from here

Phase 2 — Verify & harden (do this FIRST, before new features)

The code is written but unproven against a live backend. Close that gap before building on top.

Run it against your real Firebase project. Fill `.env.local` with the web config, `npm run dev`, and click through every screen. Compiling ≠ working — Firestore query shapes, missing indexes, and rules only surface at runtime.

Deploy rules + indexes and run the seed script. `firebase deploy --only`

`firestore:rules,firestore:indexes,storage` then `npm run seed`. Watch for index errors (Firestore tells you the exact composite index to add).

Test the security rules for real. Log in as the seeded employee and confirm they *cannot* read another employee's salary/leave; log in as admin and confirm they can. Rules are your only access control now — prove they work.

Exercise the leave-approval transaction with real data — approve a request, confirm the balance decremented exactly once and can't go negative.

Decide on tests. At minimum add a few smoke tests for the auth flow and the leave transaction. Full suite parity is optional; zero tests on an HR app long-term is a risk.

Clean up — remove `old-coriander-code/` from the repo (keep it in the old zip if you want a reference). It's 76 MB of dead weight in every clone.

Phase 3 — Add the backend (`functions/`) — when a real need hits

Most likely trigger: **custom claims for roles**. Steps:

`firebase init functions` (TypeScript) — creates `functions/` with its own `package.json`.

Add a `functions` block to `firebase.json` so it deploys with everything else.

Write the first callable/trigger you actually need — probably an `onCreate`/admin-invoked function that sets the `role` (and `employeeId`) custom claim when a user is linked to an employee. Your security rules *already reference* `request.auth.token.role` and `request.auth.token.employeeId`, so this closes a loop the rules assume exists.

Migrate the heaviest `pageAPI` fan-outs (admin dashboard aggregates) to callable functions only if client-side reads feel slow.

Everything still ships with one `npm run deploy`.

Phase 4 — Rebrand

Cosmetic and isolated — safe now that the data layer is proven. Swap the two Tailwind palettes + the Ant Design theme token block, drop in the new logo (you've already renamed to "Kora"), sweep remaining "cori"/"coriander" strings. Low risk, satisfying.

Phase 5 — The mobile employee experience (the original goal)

Now it sits on a stable, branded foundation. This is a UI-layer pass on the employee side only:

Replace Ant Design `<Table>`s with card layouts on phones (tables are the main mobile pain point).

Swap the fixed sidebar for a bottom nav or drawer on small screens.

Real responsive pass on `EmployeeHome`, `EmployeeMeetings`, `EmployeeLeaveOverview`, `EmployeeProfile`.

Cash in `onSnapshot`: now that you're on Firestore, make the employee's meeting requests and leave statuses live-update without a refresh — near-free and exactly the kind of "better phone thing" you wanted.

Optional: wrap as a PWA so employees can install it to their home screen.

The throughline

Verify → backend (if needed) → beautify → mobilize. Phase 1 built the machine; phase 2 proves it runs; then you add server-side muscle only where needed, rebrand, and finally deliver the mobile experience that motivated the whole project — each step on solid ground from the last.

What NOT to do next

Don't jump straight to mobile/rebrand while the Firebase backend is unproven against a live project — a runtime rules or index bug is far cheaper to fix now than after you've restyled everything on top of it.

Don't add `functions/` "just because" — add it when custom claims or heavy aggregates actually require it.

Don't try to share one `node_modules` between `src/` and `functions/` — they're different runtimes.